Post Processing: Project Report

Authors: Atli Arnarsson, Drake Ma, Henry Morin, Andrew Petersen

Abstract:

The project's goal is to supply post-processing effects that can be freely applied to the ray tracer's finalized image buffer. Post-processing effects may use the camera and/or depth, normal, motion, and/or position maps generated during raytracing to generate the final image:

Depth map: A buffer where each element is the number of units travelled by the ray traced from the pixel, until it hits a shape.

Normal map: A buffer where each element is the 3D normal vector of the shape that was hit by the ray traced from the pixel.

Motion map: A buffer where each element is the 3D velocity vector of the shape that was hit by the ray traced from the pixel.

Position map: A buffer where each element is the 3D position vector of the shape that was hit by the ray traced from the pixel

Table of Contents

Blend Modes

We implemented various methods of blending between two different images. These include: additive, and multiplicative blending, overlay, hard light, addition, subtraction.

Additive and multiplicative blending are two ways of combining two images into one. The first, additive blending, results in an image where every pixel is the result of adding the red, green, and blue channels of every pixel in the two images together. Multiplicative blending is the same, but they multiply the red, green, and blue channels together.

How to run

The blend modes can all be found in the `filtes/BasicEffects.h` header. These are used by passing them into each other to combine them along with other effects. For example:

```
// example
effect = Multiply(
       Image,
       Add(
              NoOp(DepthBuffer),
              NoOp(NormalBuffer)
       )
)
// effect cannot now be used
```

To make these easier to test, two buffer generators called hsin() and vsin() exist to create basic images that can help test these.

Accomplishments

N/A

State-of-the-art

These blend modes are all fairly standard in any graphics program for image manipulation or video editing. I am not aware of any state of the art for blend modes beyond having a few more functions.

Results

The blend modes that we tested had the expected behavior when tested, although it is important to make sure the ranges of inputs are understood before using them.

What We Learned

Blend Modes, and other basic filters (e.g. threshold) are relatively easy to implement, and generally follow the same pattern. Since they follow the same pattern, writing a helper to handle this code duplication for us increased the pace at which we could develop blend modes, buffer generators, and basic filters. Additionally, having these basic filters and blend modes increased development speed as it allows combining functions to be done with as little as a line of code.

Next Steps

N/A

Other Details

N/A

Hue Shift

Hue shifting is a common effect available in most image editing software, such as Photoshop. It works by converting every pixel to HSV, HSL, or some other hue-based color space, and then rotating the hue by the user's specified value.

How to run

The class that implements hue shifting is called HueShift. In it's init function, there is 1 parameter- the amount to shift by.

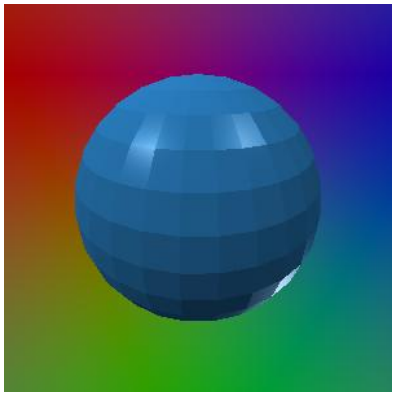
Accomplishments

N/A

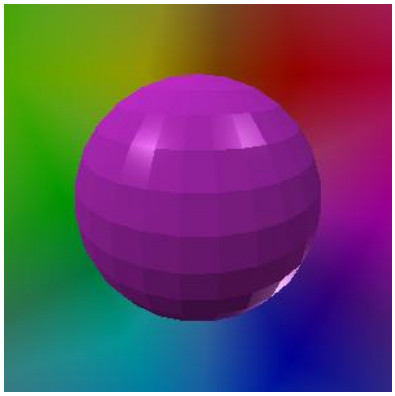
State-of-the-art

N/A

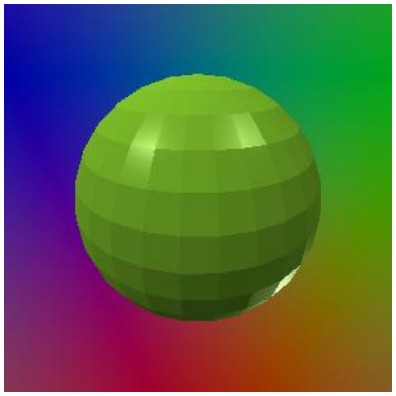
Results



Original image



Hue shifting by 90 degrees



Hue shifting by 240 degrees

What We Learned

I learned how to convert a color from RGB to HSV.

Next Steps

N/A

Other Details

N/A

Vignette

Vignette is a postprocessing effect that mimics a common artifact of photography. Photos, especially older ones, tend to fade at the edges because of the curved shape of the lens. It is a basic effect included in many photo editing softwares, such as GIMP, and many postprocessing pipeline designers, such as the one built into Unity.

How to run

The effect class that implements the vignette effect is called Vignette.It takes 5 arguments:

vignette_color, intensity, roundness, and smoothness.

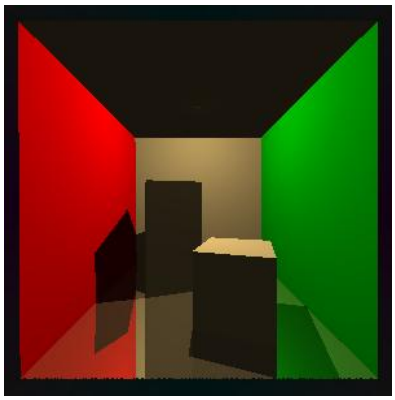
Accomplishments

The vignette class allows the developer to apply a vignette with a large amount of control over its appearance. Vignette color changes which color the edges fade to. Intensity makes the vignette circle smaller, growing inwards toward the center of the screen. Smoothness makes the fade occur slower- lower values make the border between the un-faded and faded areas sharper. Roundness changes the shape of the vignette- higher values weigh the corners of the image less and the edges more. Lower values do the opposite- setting roundness to 0.5 makes the vignette take on a diamond shape, with the corners faded and the edges unfaded.

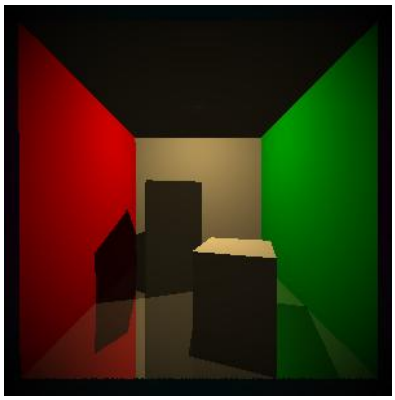
State-of-the-art

N/A

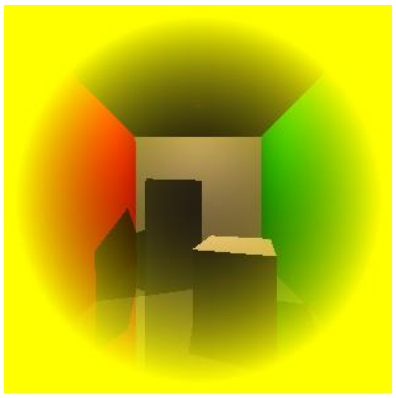
Results



Original image



Vignette with default parameters



Vignette with yellow vignette_color, smoothness=0.5, and power=1.5

What We Learned

I didn't learn much from implementing this, I mostly used it as a test of the pipeline. I had already known generally how the effect worked, it was just a matter of finding a source for the equation and translating it to C++.

Next Steps

If I were to add anything else, I would allow users to supply a custom image for the vignette.

Other Details

N/A

Ordered and Floyd-Steinberg Dithering

Description

Dithering is a set of techniques in post processing traditionally used to reduce file size. It does so by reducing the size of the color palette in the image while applying analytic methods to preserve details. In both ordered and Floyd-steinberg dithering, the programmer selects a desired color palette. Every pixel in the resulting image will have a color chosen from the palette.

How to run

To add Bayerian dithering to the pipeline, use:

```
(new BayerianDither(<effect1: Effect, effect2: Effect>))->init(<palette: vector<Vector3>>)
```

Running init is optional- if it is not called, a color palette of 64 equally spaced colors will be used as the palette.

To add Floyd-Steinberg dithering to the pipeline: use:

```
(new BayerianDither(<effect1: Effect, effect2: Effect>))->init(<useGlitch: bool>)-
>setPallette(<pallette: vector<Vector3>>)
```

Once again, init is optional- useGlitch defaults to false. Setting it to true results in an effect that I thought looked cool. setPallette is also optional- if it is not called, a color palette of 64 equally spaced colors will be used as the palette.

Accomplishments

Ordered, or Bayerian dithering, uses a matrix called the threshold map to adjust the pixels in the image to “produce the greatest impression of the underlying feature”. This matrix is essentially tiled over the image, blended additively. The result of this is quantized with the color palette to produce the dithered result. This quantization consists of replacing each pixel with the closest color in the color palette to it. In implementing this, I referenced the Wikipedia page on the algorithm.

Floyd-Steinberg dithering is a kind of error diffusion method in dithering. Rather than adjusting each pixel according to a precalculated threshold map, every pixel is adjusted

according to how far off it's neighbor pixels are from the quantized pixel. The pseudocode for Floyd-Steinberg dithering is as follows:

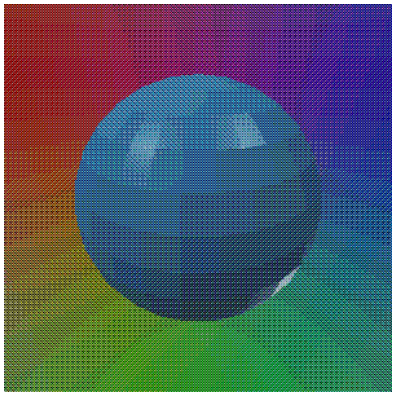
```
for x in image; for y in image do
   pixel = image[x,y]
   quantized_color = closestColor(pixel)
   image[x,y] = quantized_color
   quantization_error = pixel − quantized_color
   image[x+1,y     ] += error * 7/16
   image[x-1, y+1] += error * 3/16
   image[x    , y+1] += error * 5/16
   image[x+1, y+1] += error * 1/16
```

Because each pixel only affects those to the right and below it, the top row and left column of the image is the same as if it had been quantized. In implementing this algorith, I referenced the Wikipedia page on it .

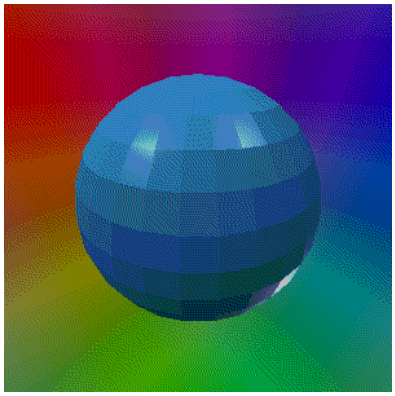
State-of-the-art

These algorithms have both been improved upon. My closest-color algorithm is a naïve one. A better algorithm uses nearest-neighbor search in 3D. The individual algorithms as well better preserve details in the image. My ordered dithering algorithm can be improved with a larger threshold map, and by generating a threshold map specific to the palette used.

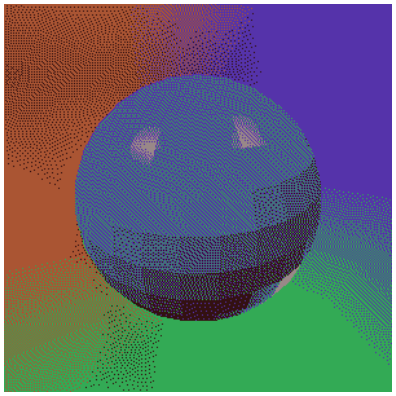
Results



Bayerian dithering applied to the blue sphere scene with default color palette.



Floyd-Steinberg dithering applied to the blue sphere scene with default color palette.



Floyd-Steinberg dithering applied to the blue sphere scene with a color palette of just 5 colors.

What We Learned

I learned that palette is spelled with one L and two Ts, for some reason. Perhaps more importantly, I learned about these two approaches to dithering. In the past, I had considered dithering to be one distinct algorithm, perhaps with a few simple variations. I learned that dithering is commonly used in audio production to solve a similar problem- audio frequency banding, instead of color banding.

Next Steps

My next steps in improving it would be to improve the closest color function and implement some of the improved dithering algorithms.

Other Details

N/A

Motion Blur

This effect applies bi-directional blur to objects in the scene as if they're in motion.

How to run

Go to **`PostProcessor.h`** and search for **`DefaultPipeline`** on line 62. Replace the content of the **`buildPipeline()`** method with the following:

```
return new PostProcessor(ConstMultiply(new RGBConvert(new ToneMapHSV(new
HSVConvert((new MotionBlur(imageBuffer))->init(motionBuffer)))), 255.0f));
```

To modify the velocity for certain objects in the scene, go to `ObjLoader.h`. To change the spheres' velocity, head to **line 31**. To change the triangles' velocity, head to **line 51**. To change the camera's velocity, head to **line 128** (4th parameter of the `Camera` constructor).

Accomplishments

This effect is able to accurately portray moving objects in the scene with appropriate blurs. This is done by performing the following steps:

1. Assign each shape with a velocity based on their material
2. For each ray that hits a shape: (`RayTracer.h`)
    a. The `Hitpoint` receives velocity from the object
    b. Calculate the final 3D motion by subtracting the camera velocity from the object velocity
    c. Project the 3D motion in the direction of ray so that `x` and `y` are in terms of camera's 2D space
    d. Negate `y` of the newly calculated 2D motion
    e. Scale the 2D motion by the focal length of the camera and the depth of the ray (parameter)
    f. Put the 2D motion inside the motion map
3. Dilate the motion map (inspired by McGuire's 2012 paper) (`MotionBuffer.h`)
    a. Divides the motion map into chunks of 8x8 pixel size (customizable)
    b. For each chunk, iterate over each pixel inside the chunk to get the motion with the highest magnitude
    c. For each chunk, iterate over its neighboring chunks to get the motion with the highest magnitude
    d. For each pixel in each chunk, sample 32 points (customizable) along the chunk motion and then average it to get the final motion for the pixel

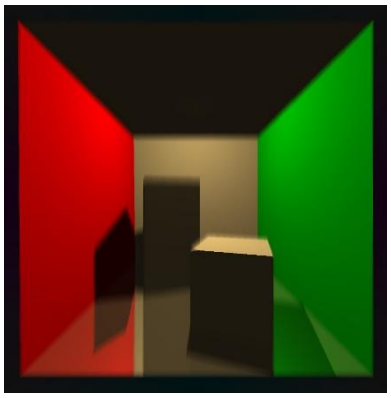
4. Blur the image using the dilated motion map (`MotionBlur.h`)
    a. For each pixel, sample 32 points (customizable) along the pixel motion in the image buffer and average these colors to get the final pixel color

State-of-the-art

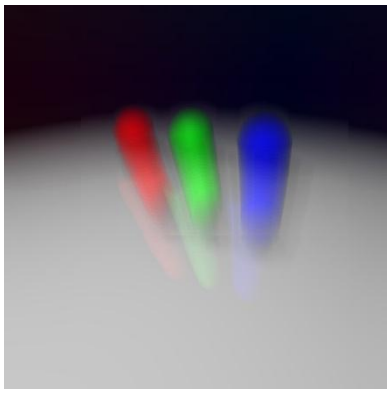
- Hybrid Rendering: MBlur
    - When moving objects are covered by static foreground objects, there may be inaccuracies in how the blur is portrayed
    - MBlur uses another pass of ray tracing to get data on each pixel to calculate the final blur needed

Results

Stanford bunny with a velocity of **<0, 0, 1>** (moving parallel to the camera's line of sight)



Cornell box, both the camera and the short box have velocity of **<0, 50, 0>**



Spheres scene, camera moving **up** *(we forgot what the velocity was)*

What We Learned

Motion blur as a post process effect can be cheap to calculate but requires advanced techniques to match the accuracy of realistic motion blur. A more realistic motion blur can be done by tracing the scene multiple times, with each frame containing objects that move very slightly based on delta (time difference between each frame). However, that is very time consuming, so post process motion blur is used more commonly for real-time applications.

Next Steps

Currently, if a static foreground covers a moving object, the blend will bleed into the foreground object, which is not ideal. It may be better if we somehow compare the final motion map with the depth map and modify each motion based on depth differences. However, that is only a guess so it might not work out as intended. MBlur is also something worth looking into and would be useful to implement.

Other Details

N/A

Daltonization

This effect reduces color confusion for people with Color Vision Deficiency (CVD) by enhancing the color they can see clearly and reducing the color they often confuse with. This effect can also simulate CVD and the color enhanced image in their perspective.

How to run

Go to **`PostProcessor.h`** and search for **`DefaultPipeline`** on line **62**. Replace the content of the **`buildPipeline()`** method with the following:

```
CVDType cvd_type = CVDType::PROTANOPIA;
bool compensate = true;
bool simulate = false;
return new PostProcessor(ConstMultiply(new RGBConvert(new ToneMapHSV(new
HSVConvert((new Daltonization(imageBuffer))->init(cvd_type, compensate,
simulate)))), 255.0f));
```

By default, `compensate` is set to `true` and `simulate` is set to `false` in the Daltonization initialization. However, they are explicitly defined here for demonstration purposes. There are 3 types of CVD implemented here: Protanopia, Deuteranopia, and Tritanopia. If `compensate` is set to `true`, the image will be recolored to assist the person with the designated CVD. If `simulate` is set to `true`, the image will be recolored to imitate the vision of the person with the designated CVD. If **both** options are set to `false`, nothing is recolored. If **both** options are set to `true`, the image will be recolored to imitate what the person with the designated CVD will see after the image is enhanced for their condition.

It is recommended to run the ishihara test scene as well for a more immersive experience:

```
>>> ./tracer -r 400 400 ../../tests/ishihara9.svg
```

Accomplishments

This effect is able to allow people with CVD to correctly distinguish commonly confused colors from each other when looking at the Ishihara test. This is testable since Drake and Henry have mild color blindness. Assuming `compensate` is set to `true` and `simulate` is set to `false`, the recoloring process would be as follows:

For each pixel:

1. Convert the RGB vector into a LMS (Long cone, Medium cone, Short cone) vector. This categorizes colors into their respective wavelengths:

$$\begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix} = \begin{bmatrix} 17.8824 & 43.5161 & 4.11935 \\ 3.45565 & 27.1554 & 3.86714 \\ 0.0299566 & 0.184309 & 1.46709 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

2. Apply CVD to LMS
   a. Protanopia:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 0 & 2.02344 & -2.52581 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix}$$

   b. Deuteranopia:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0.494207 & 0 & 1.24827 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix}$$

   c. Tritanopia:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0.395913 & 0.801109 & 0 \end{bmatrix} \begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix}$$

3. Convert LMS back to RGB:

$$\begin{bmatrix} R_n \\ G_n \\ B_n \end{bmatrix} = \begin{bmatrix} 0.0809444479 & -0.130504409 & 0.116721066 \\ -0.0102485335 & 0.0540193266 & -0.113614708 \\ -0.000365296938 & -0.00412161469 & 0.693511405 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix}$$

4. Subtract this new RGB from the original RGB to get the `error` vector (the color that the person with CVD would have trouble seeing):

   $RGB_e = RGB - RGB_n$

5. Calculate `fix` by shifting `error` towards the visible spectrum:

$$\begin{bmatrix} R_a \\ G_a \\ B_a \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0.7 & 1 & 0 \\ 0.7 & 0 & 1 \end{bmatrix} \begin{bmatrix} R_e \\ G_e \\ B_e \end{bmatrix}$$

6. Add `fix` to original RGB to get final pixel color:

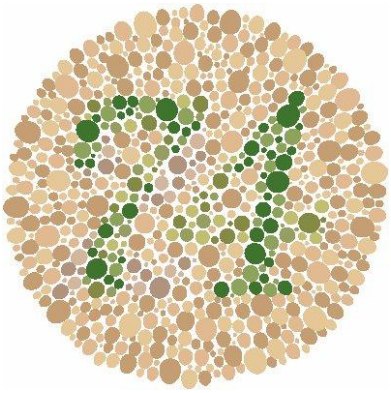
   $RGB \mathrel{+}= RGB_a$

This process is inspired by Tecson et al.'s research paper.

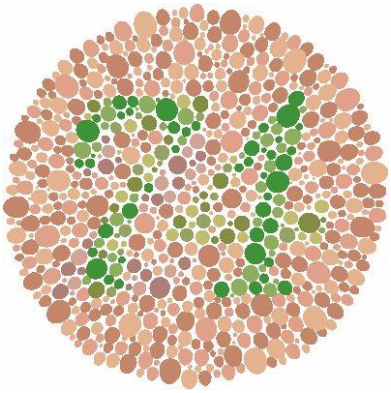
State-of-the-art

There are many types of daltonization methods available, each with their own strengths and weaknesses. Some notable methods include Kotera's Method, Machado's Method, Hoffman's Method, and Ebelin's Method. Methods with complex algorithms that aim for accuracy may not be suitable for real-time systems, for example. Ebelin's method was an interesting one, though the research paper provided is rather vague and it is difficult to follow through.

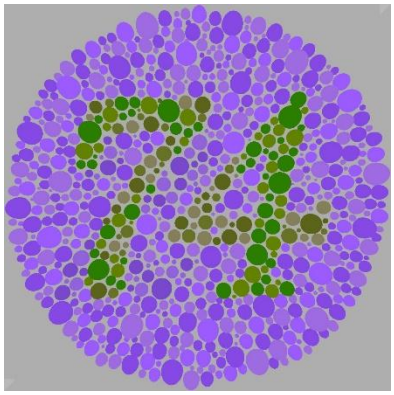
Results



Fixed for Protanopia



Fixed for Deuteranopia



Fixed for Tritanopia

What We Learned

All the daltonization methods serve in their own niche fields. The method implemented in this ray tracer is the most used one, and also possibly the simplest. Due to its simplicity, it can provide a general value to our final image output without requiring specific conditions.

Next Steps

Implement Ebelin's method, as it's the most applicable to what we're doing real-time wise.

Other Details

A custom parser was made from scratch to convert Ishihara SVG to OBJ/MTL files (`ishihara_svg_parser.h`). This parser currently only works for `ishihara9.svg`, because this parser makes spheres based on the `<circle>` tag. Other ishihara tests are either not in SVG form or use `<path>` instead of `<circle>`.

Blur

Description

A standard separable convolution Gaussian Blur. It reduces the clarity and sharpness of the provided image by a standard amount.

How to run

To run the standard blur kernel on the pipeline, use:

```
(new BasicConvolution(<effect1: Effect>))->init(<passes: int>)
```

Running init is optional- if it is not called, blur will be applied once.

To run a different convolution kernel on the pipeline, use:

```
(new BasicConvolution(<effect1: Effect>))->init(<passes: int>, <kernalX:
Vector3<float>>, <kernalY: Vector3<float>>)
```

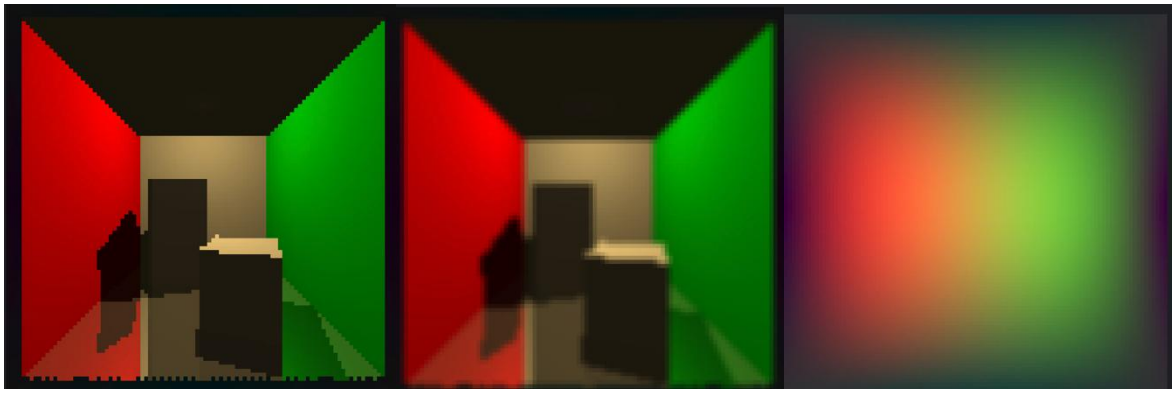
Accomplishments

This is a basic implementation of a separable convolution using our pipeline system for effects. It takes the image buffer that was returned by the effect that was passed to it and replaces each pixel with a function of the pixels surrounding it. Since this is a separable convolution, it does the calculations in two separate passes. One to multiply in the X direction and one to multiply in the Y direction. This provides a significant compute benefit at the disadvantage of using twice as much memory. The effect itself was designed to take alternative kernels as optional parameters so the algorithm can be reused for additional effects.

State-of-the-art

These algorithms seem to be standard. The only arguably better options are approximations of the same effects.

Results



Difference between no blur (left), one pass of blur (middle), and 1000 passes of blur (right).

What We Learned

We learned all about convolution and how different kernels can affect the output image. We also learned about the differences between regular convolution and separable convolution and how some kernels cannot be separated.

Next Steps

It would be interesting to see how other kernels could produce useful or artistic result.

Edge Detect

Description

Edge detection effects highlight the edges of objects in a scene while darkening the rest of the scene. Typically, a good edge detection would identify all edges and turn the rest of the image completely dark. This effect is more artistic than practical as this goal has no basis in reality.

How to run

To run the standard blur kernel on the pipeline, use:

```
(new BasicConvolution(<effect1: Effect>))
```

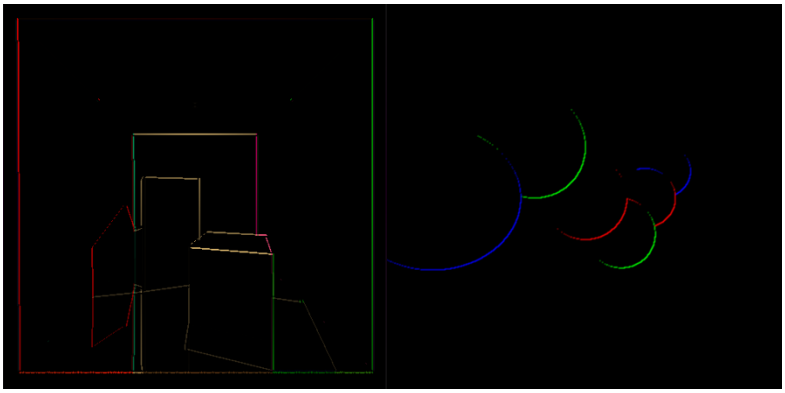
Accomplishments

To build this effect two different convolution kernels are used. The specific kernels are commonly known as the Sobel Filter. There is one for the X axis and one for the Y axis. After both convolutions are run, the two images are added together, and the resulting image includes only the detected edges.

State-of-the-art

This effect can be done many ways including the way done here. A more accurate algorithm might take the normal or depth map and use a more complicated algorithm that produces better results. As this is an artistic effect, there is no definite state-of-the-art and it all comes down to the desired visual outcome of the effect.

Results



What We Learned

We learned more about what you can accomplish with different convolution kernels. This effect was useful to help us test our addition combiner as well.

Next Steps

It would be interesting to continue to build on the edge detection concept without a convolution algorithm and use the depth map to detect when objects suddenly drop off.

SSAO

Screen Space Ambient Occlusion (SSAO) is a method of approximating Ambient Occlusion far faster by using the depth buffer (or hit position buffer) to approximate the calculations of ambient occlusion. Ambient occlusion measures what fractions of angles are blocked from receiving light to approximate how much ambient light a position should receive. SSAO tries to approximate this by using data from the depth or position buffer.

How to run

To run this, use the SSAO Effect, however, on its wont his will not work achieve the intended effect on its own. The SSAO class is misleadingly named, instead of calculating the entire occlusion buffer or applying it to the input, it returns a buffer of how occluded it believes every point is. To use this properly it makes sense to blur it and then add or subtract (depending on if it returns how occluded every pixel is inverted) it

Accomplishments

Was able to achieve a relatively accurate Ambient Occlusion effect using just post processing information.

State-of-the-art

SSAO is used in many modern games, but as computing power increases, it may make sense to calculate actual Ambient Occlusion. However, when sticking to strict SSAO techniques, there are several different algorithms that are used to try to as fast as possible approximate occlusion with maximum accuracy. Arguably the largest benefit of SSAO is that it can be done quickly and entirely on the GPU (because it depends on the depth buffer not the actual scene complexity, which of course means the State-of-the-art means running on the GPU.

Results

(left: without SSAO, center: SSAO occlusion buffer (blurred), right: with SSAO)

What We Learned

We learned what Ambient Occlusion is and also what SSAO is. TODO

Next Steps

On next step would be to implement a more modern SSAO algorithm.

Make a helper class to do `OcclusionBuffer -> blur -> add/subtract to image` in one step.

Other Details

N/A